

Math 122a

Final project: Option A

Due: Wednesday, 5/6

Part 1 *Matrix completion by gradient descent*

In Homework #7, we used the method of alternating least squares to fill in missing movie ratings. Here, we explore a different method using gradient descent. Suppose as before we have a set m users and n movies, with a set Ω of known user/movie ratings, and a rating a_{ij} for each pair $(i, j) \in \Omega$. Recall that we discussed finding a low-rank approximation $A \approx U^T V$ to minimize the quantity

$$F(\mathbf{u}_1, \dots, \mathbf{u}_m, \mathbf{v}_1, \dots, \mathbf{v}_n) = \sum_{(i,j) \in \Omega} (a_{ij} - \mathbf{u}_i^T \mathbf{v}_j)^2$$

where the $\mathbf{u}_i$ denote rows of U and the $\mathbf{v}_j$ denote columns of V . (Note, that this is equivalent to the alternating least squares formulation discussed previously, upon switching the labeling from rows/columns of U and V .) Your task is to implement gradient descent for the MovieLens dataset. Use the file `ratings-train.csv` to fit your model, and the file `ratings-test.csv` to evaluate it.

- (i) Calculate (symbolically by hand, not numerically in Python) the gradient of the expression

$$\sum_{(i,j) \in \Omega} (a_{ij} - \mathbf{u}_i^T \mathbf{v}_j)^2$$

with respect to the vector $\mathbf{u}_i$ for a fixed user, and with respect to the vector $\mathbf{v}_j$ for a fixed movie. Notice that if we observe all entries (i.e. in the case where $\Omega = \{\text{all possible pairs } (i, j)\}$), this is just the Frobenius norm,

$$\|A - U^T V\|_F^2,$$

where U has a row corresponding to each user (you can think of these as user features) and V has columns correspond to each movie (think of this as the parameters for each movie).

- (ii) Sketch in pseudocode the gradient descent algorithm for fitting the model described above. Give two variants: with the gradients averaged over the entire

dataset at each iteration, and with only a single example used to estimate the gradient at each iteration (i.e., stochastic gradient descent; a batch size of one). In other words, for the stochastic gradient method, at each iteration you randomly select i and j and update $\mathbf{u}_i$ and $\mathbf{v}_j$ by using gradients of the function $\tilde{F}(\mathbf{u}_i, \mathbf{v}_j) = (a_{ij} - \mathbf{u}_i^T \mathbf{v}_j)^2$.

- (iii) Set $k = 10$ and step size $\alpha = 0.01$ and implement both versions of gradient descent you sketched above. Evaluate the performance of the resulting model on the test dataset. What differences do you observe between the two variants on the optimization procedure? Do they take different amounts of time?

.....

Part 2 *Open-ended exploration*

Go beyond your findings in Part 1 to explore a question of interest to your group. For example, you could apply the method to a different dataset or propose a modified approach and compare the results. Prepare a short (4-8 minute) video sharing your findings. No particular format is required—be creative and try to have fun!

.....